

Le dilemme du code legacy

Maintenir ou Réécrire ?

A low-angle photograph of a modern glass skyscraper. The Equifax logo, consisting of the word "EQUIFAX" in large, red, three-dimensional block letters, is mounted on a horizontal metal beam extending from the building's facade. The building's glass windows reflect the sky and surrounding environment. Some green foliage is visible on the right side of the frame.

EQUIFAX

Causes de la faille

 (CVE-2017-5638) dans **Apache Struts** : Correctif en mars 2017, non appliqué

 **Corrections tardives** : Audit infructueux, certificats TLS expirés, complexité de code

 Pas de **segmentation réseau**: Accès aux bases de données sur l'ensemble du réseau

 **Aucune limite** de requête BDD: Extraction de données sensibles sans détection

Ce qui aurait pu fonctionner ?



Appliquer les correctifs **dès leur publication**



Renforcer l'authentification : **authentification forte** et **multifactorielle** pour tous les accès administratifs



Monitorer et limiter les **requêtes** BDD : requêtes excessives



Amélioration continue sur les process et les méthodes de développement



Valentin DUMAS

Ingénieur logiciel



“ Legacy code is simply code **without tests** ”

— Michael Feathers

“ Legacy code is **valuable** code
you're **afraid** to change ”

— Nicolas Carlo & Alex Bolboaca

Extrait du Gilded Rose refactoring Kata

```
1  public void updateQuality() {
2      for (int i = 0; i < items.length; i++) {
3          if (!items[i].name.equals("Aged Brie")
4              && !items[i].name.equals("Backstage passes to a TAFKAL80ETC concert")) {
5              if (items[i].quality > 0) {
6                  if (!items[i].name.equals("Sulfuras, Hand of Ragnaros")) {
7                      items[i].quality = items[i].quality - 1;
8                  }
9              }
10         } else {
11             if (items[i].quality < 50) {
12                 items[i].quality = items[i].quality + 1;
13
14                 if (items[i].name.equals("Backstage passes to a TAFKAL80ETC concert")) {
15                     if (items[i].sellIn < 11) {
16                         if (items[i].quality < 50) {
17                             items[i].quality = items[i].quality + 1;
```

Un peu de legacy

Enjeux et challenges

Un peu de legacy

Enjeux et challenges



Un peu de legacy

Enjeux et challenges



Un peu de legacy

Enjeux et challenges



Un peu de legacy

Enjeux et challenges



> 100k lignes

Un peu de legacy

Enjeux et challenges



> 100k lignes



Un peu de legacy

Enjeux et challenges

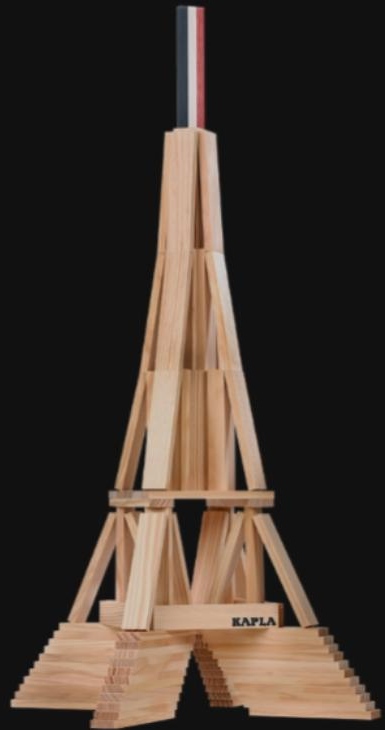


> 100k lignes



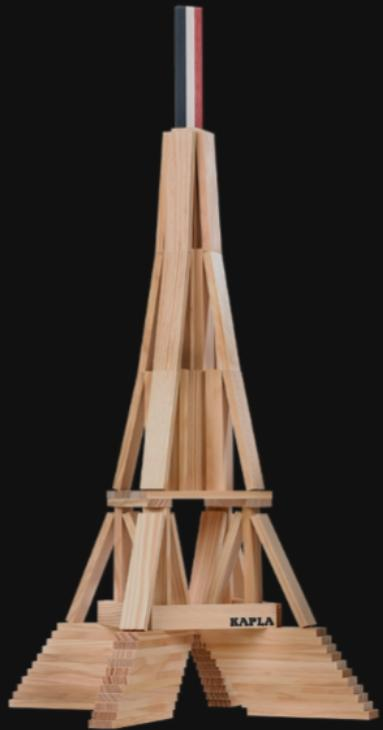
Un peu de legacy

Enjeux et challenges



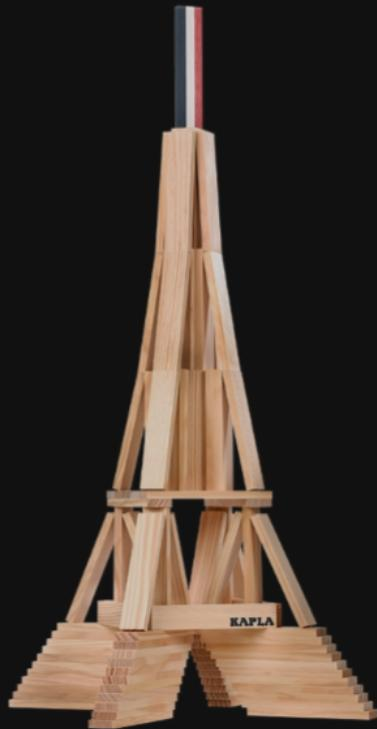
Un peu de legacy

Enjeux et challenges



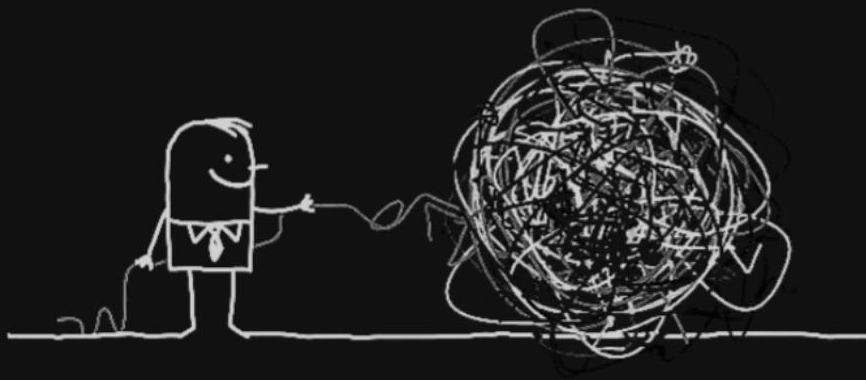
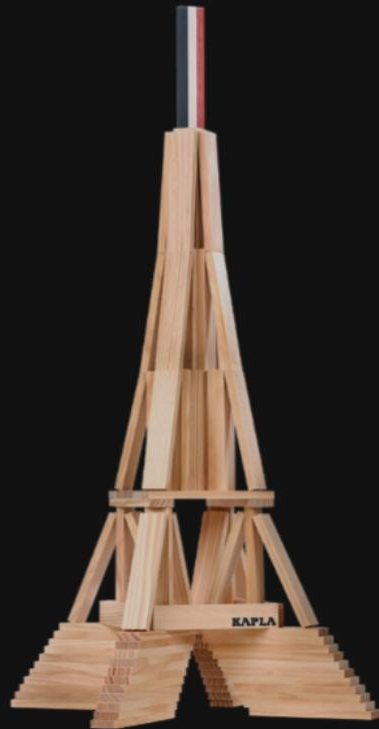
Un peu de legacy

Enjeux et challenges



Un peu de legacy

Enjeux et challenges



“ Tout code **deviendra legacy** ”

Symptômes d'un code legacy

Comprendre les sources des problèmes

Symptômes d'un code legacy

Comprendre les sources des problèmes



Facteurs externes

Symptômes d'un code legacy

Comprendre les sources des problèmes

Facteurs externes

- Évolution du modèle
- Changements réglementaires
- Turn-over
- >> users que prévu
- Pression business pour livrer vite
- Empilement de demandes clients

Symptômes d'un code legacy

Comprendre les sources des problèmes



Facteurs externes

- Évolution du modèle
- Changements réglementaires
- Turn-over
- >> users que prévu
- Pression business pour livrer vite
- Empilement de demandes clients



Facteurs internes

Symptômes d'un code legacy

Comprendre les sources des problèmes

Facteurs externes

- Évolution du modèle
- Changements réglementaires
- Turn-over
- >> users que prévu
- Pression business pour livrer vite
- Empilement de demandes clients

Facteurs internes

- Vitesse basse
- Bugs et regressions
- Connaissance / 1 personne
- Couplage fort entre modules
- Absence ou faible couverture de tests
- Manque de documentation

Problèmes pour les devs

 Rechignement

 Tensions

 Résignation

Problèmes pour les entreprises

 Perte de compétitivité

 "Bus factor"



Situations fréquentes vs



Solutions



Situation



Solution



Situations fréquentes vs



Solutions



Situation



Solution

Pas le temps / pas le budget

Perte de maîtrise: risques vs coûts

Incompréhension du code



Présenter les **risques concrets** et les **coûts potentiels**



Situations fréquentes vs



Solutions

🔗 Situation

🔧 Solution

Pas le temps / pas le budget

Perte de maîtrise: risques vs coûts

Incompréhension du code

👉 Présenter les **risques concrets** et les **coûts potentiels**

Refonte en sous-marin

Estimation faussée volontairement

Confiance rompue entre devs et produit.

💛 Recréer la **confiance** entre les équipes.



Situations fréquentes vs



Solutions

🌱 Situation

Pas le temps / pas le budget

Perte de maîtrise: risques vs coûts

Incompréhension du code

Refonte en sous-marin

Estimation faussée volontairement

Confiance rompue entre devs et produit.

Refonte sans fin

Deux codebases à maintenir. La nouvelle devient aussi legacy.

Aucun plan clair.

🔧 Solution

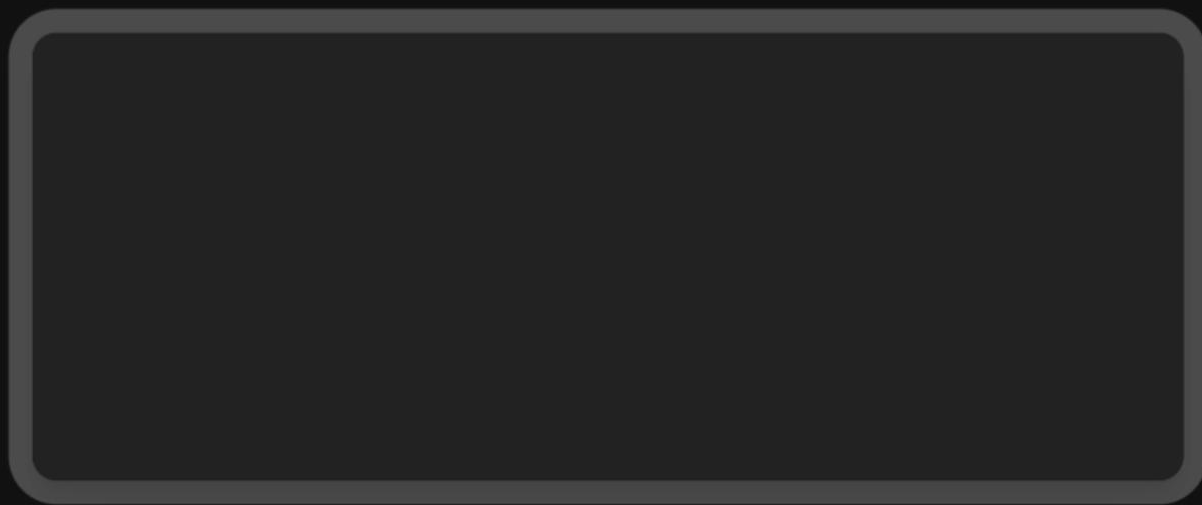
👉 Présenter les **risques concrets** et les **coûts potentiels**

💛 Recréer la **confiance** entre les équipes.

🕒 Besoin de **planification** claire

📊 **Former** au refacto

Etude de cas



Etude de cas

 Banque en ligne française qui veut s'ouvrir à l'international

Etude de cas

 Banque en ligne française qui veut s'ouvrir à l'international

 Besoin: gestion multidevises

Etude de cas

 Banque en ligne française qui veut s'ouvrir à l'international

 Besoin: gestion multidevises

 Code complexe et fragile

Etude de cas

 Banque en ligne française qui veut s'ouvrir à l'international

 Besoin: gestion multidevises

 Code complexe et fragile

 Compréhension difficile

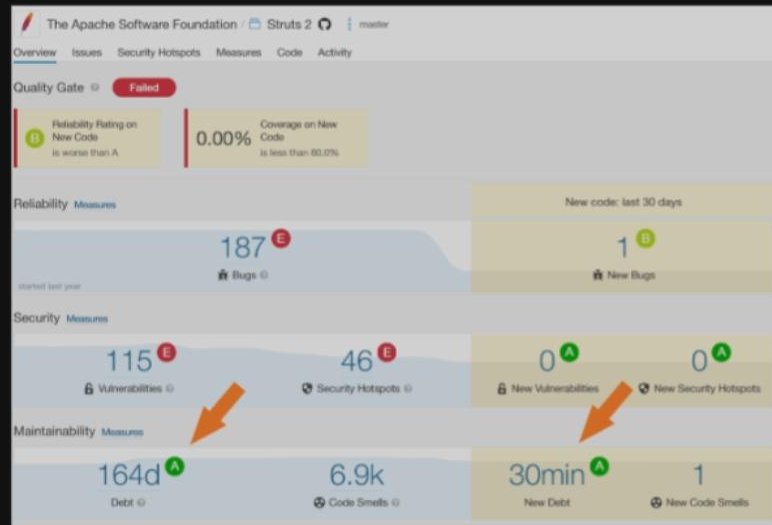
Mesurer les risques et les coûts

Mesurer les risques et les coûts

🌸 **Qualité de code** (dette, code smells, WTF par minute)

Mesurer les risques et les coûts

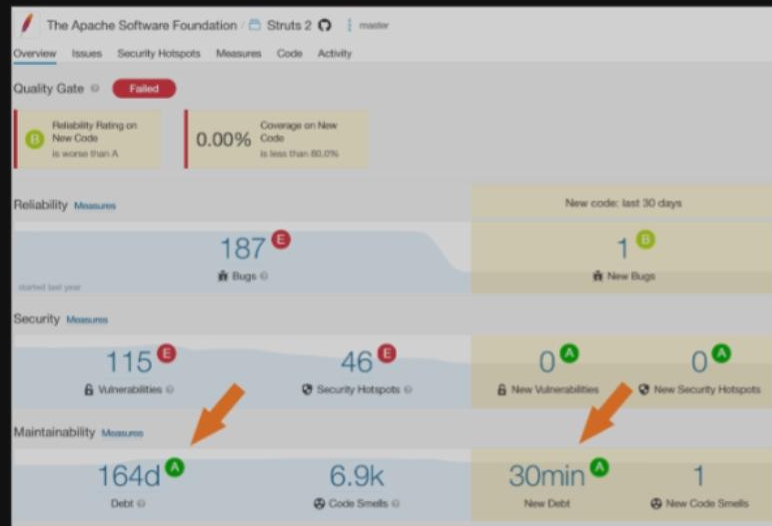
 **Qualité de code** (dette, code smells, WTF par minute)



Mesurer les risques et les coûts

 **Qualité de code** (dette, code smells, WTF par minute)

 **Nombre de régressions fonctionnelles**

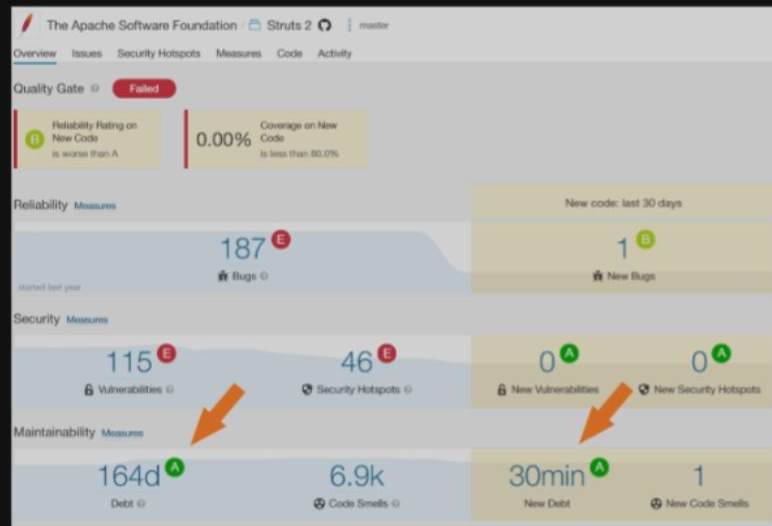


Mesurer les risques et les coûts

 **Qualité de code** (dette, code smells, WTF par minute)

 Nombre de **régressions** fonctionnelles

 Temps **estimé** vs Temps **réel**



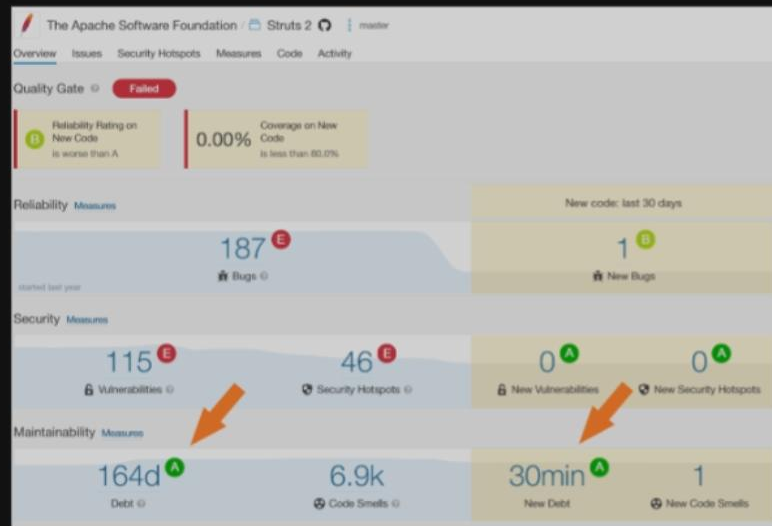
Mesurer les risques et les coûts

🌸 **Qualité de code** (dette, code smells, WTF par minute)

🐸 Nombre de **régressions** fonctionnelles

🕒 Temps **estimé** vs Temps **réel**

🌿 **Coût réel** par feature



Proposer un plan

Proposer un plan

 Périmètre d'intervention

Proposer un plan

 Périmètre d'intervention

 Définir un périmètre d'**intervention initial**

Proposer un plan

 Périmètre d'intervention

 Définir un périmètre d'**intervention initial**

 Actions

Proposer un plan

Périmètre d'intervention

 Définir un périmètre d'**intervention initial**

Actions

 Pratiques de développement

Proposer un plan

Périmètre d'intervention

 Définir un périmètre d'**intervention initial**

Actions

 Pratiques de développement

 Expliciter les concepts métier

Proposer un plan

Périmètre d'intervention

 Définir un périmètre d'**intervention initial**

Actions

 Pratiques de développement

 Expliciter les concepts métier

Ex : une colonne "montant" → en fait un montant en euros : migration, renommage, typage

Proposer un plan

Périmètre d'intervention

 Définir un périmètre d'**intervention initial**

Actions

 Pratiques de développement

 Expliciter les concepts métier

Ex : une colonne "montant" → en fait un montant en euros : migration, renommage, typage

 Eviter le "code freeze" !

Proposer un plan

Périmètre d'intervention

 Définir un périmètre d'**intervention initial**

Actions

 Pratiques de développement

 Expliciter les concepts métier

Ex : une colonne "montant" → en fait un montant en euros : migration, renommage, typage

 Eviter le "code freeze" !

Ne pas isoler la refonte sur une branche morte. Indicateur : mesure de la part de "nouveau" code réellement exécuté



Communiquer avec les parties prenantes

Se synchroniser et avancer collectivement



Communiquer avec les parties prenantes

Se synchroniser et avancer collectivement



Inform



Communiquer avec les parties prenantes

Se synchroniser et avancer collectivement



Inform



Présenter le plan de **refonte**



Communiquer avec les parties prenantes

Se synchroniser et avancer collectivement



Inform



Présenter le plan de **refonte**



Négocier la planification



Communiquer avec les parties prenantes

Se synchroniser et avancer collectivement



Inform



Présenter le plan de **refonte**



Négocier la planification



Obtenir l'**accord** de la direction



Communiquer avec les parties prenantes

Se synchroniser et avancer collectivement



Inform



Présenter le plan de **refonte**



Négocier la planification



Obtenir l'**accord** de la direction



Aligner avec la **stratégie d'entreprise**

 **Suivre** et donner de la **visibilité**

Suivre et donner de la **visibilité**

 Éviter l'**effet tunnel**

Suivre et donner de la visibilité

 Éviter l'**effet tunnel**

 Tout changement est rollback-able rapidement

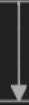
Suivre et donner de la visibilité

 Éviter l'**effet tunnel**

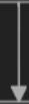
 Tout changement est rollback-able rapidement

 Partager l'**avancement** → feedback

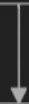
Mesurer les risques / coûts



Proposer un plan



Communiquer



Suivre & Rassurer

“ Legacy code is about the **cost of change** ”

— Michael Feathers

Un peu de réécriture

Brownfield development

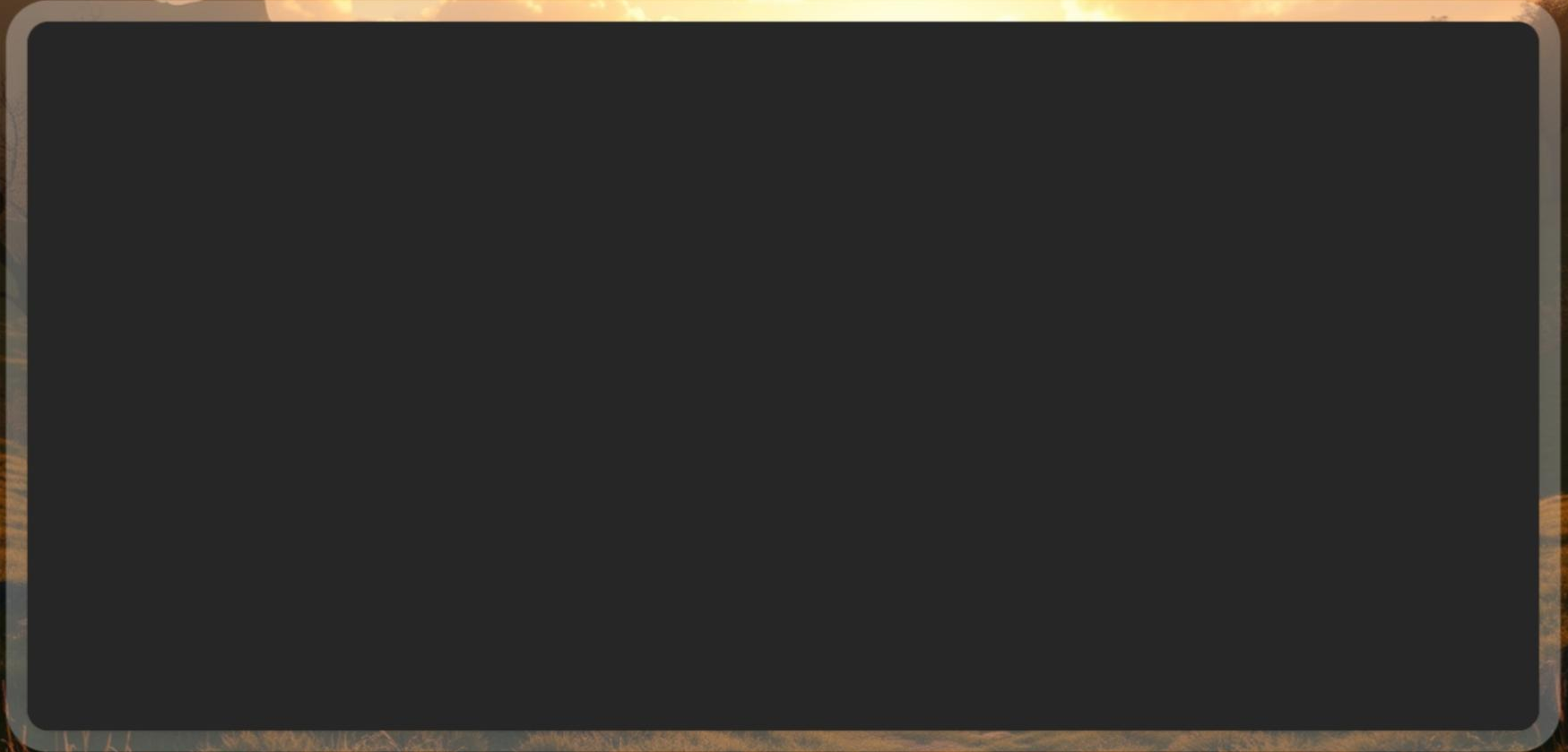
- 👉 Reprise de l'existant
- 🌿 Surprises dans le legacy
- 🔒 Stack de l'existant
- 🔥 Risques d'endettement ++ (bugs, dette, deps.)
- 🧩 Challenge: Intégrer des nouvelles features

Un peu de réécriture

Greenfield development

- ⌚ Délai initial plus court
- 💰 Budget prévisible
- 🧱 Stack technique / architecture: choix libre
- 🟢 Peu de risques / complexité (0 dette)
- 🧩 Nécessite décisions: UI, backend, infra, tests...

Prendre une décision



Prendre une décision



Réécrire

" Le code existant est un handicap (pas de tests, pas de documentation, mauvaise structure) "

" Vous disposez du temps, du budget et de la tolérance au risque nécessaires pour le reconstruire "

Prendre une décision



Réécrire

" Le code existant est un handicap (pas de tests, pas de documentation, mauvaise structure) "

" Vous disposez du temps, du budget et de la tolérance au risque nécessaires pour le reconstruire "



Rearchitect

" L'architecture de votre système limite la croissance "

" Vous passez à des services en microservices "

Prendre une décision



Réécrire

" Le code existant est un handicap (pas de tests, pas de documentation, mauvaise structure) "

" Vous disposez du temps, du budget et de la tolérance au risque nécessaires pour le reconstruire "



Rearchitect

" L'architecture de votre système limite la croissance "

" Vous passez à des services en microservices "



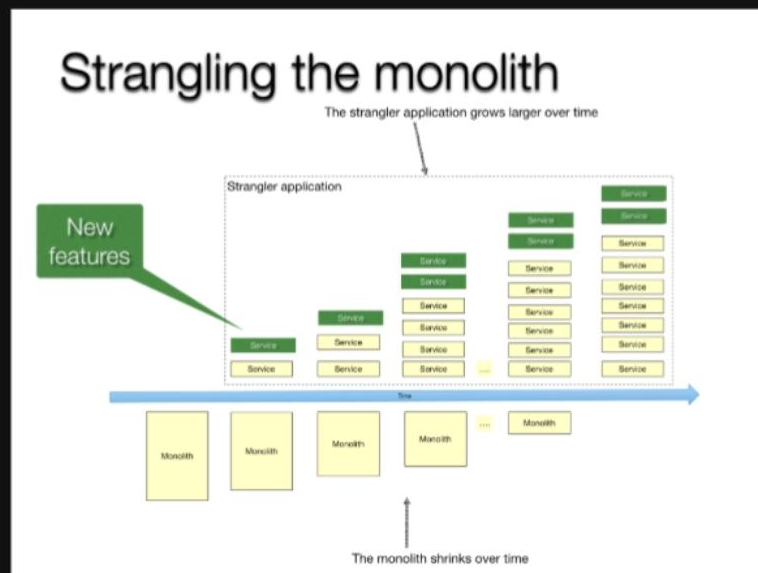
Refactor

" Vous pouvez nettoyer le code en toute sécurité et de manière incrémentale, sans trop impacter les fonctionnalités existantes "

“ On choisit de s’orienter vers
un **refactoring progressif** ”

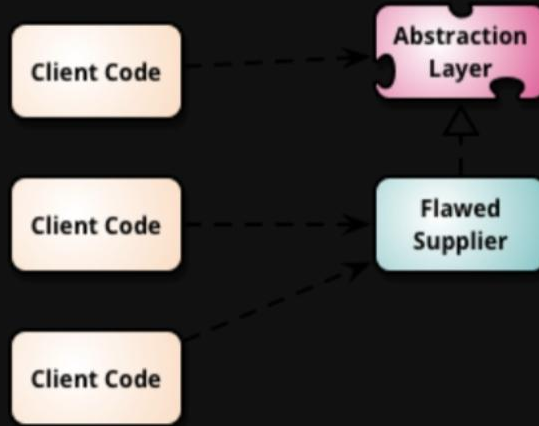
Réarchitecturer

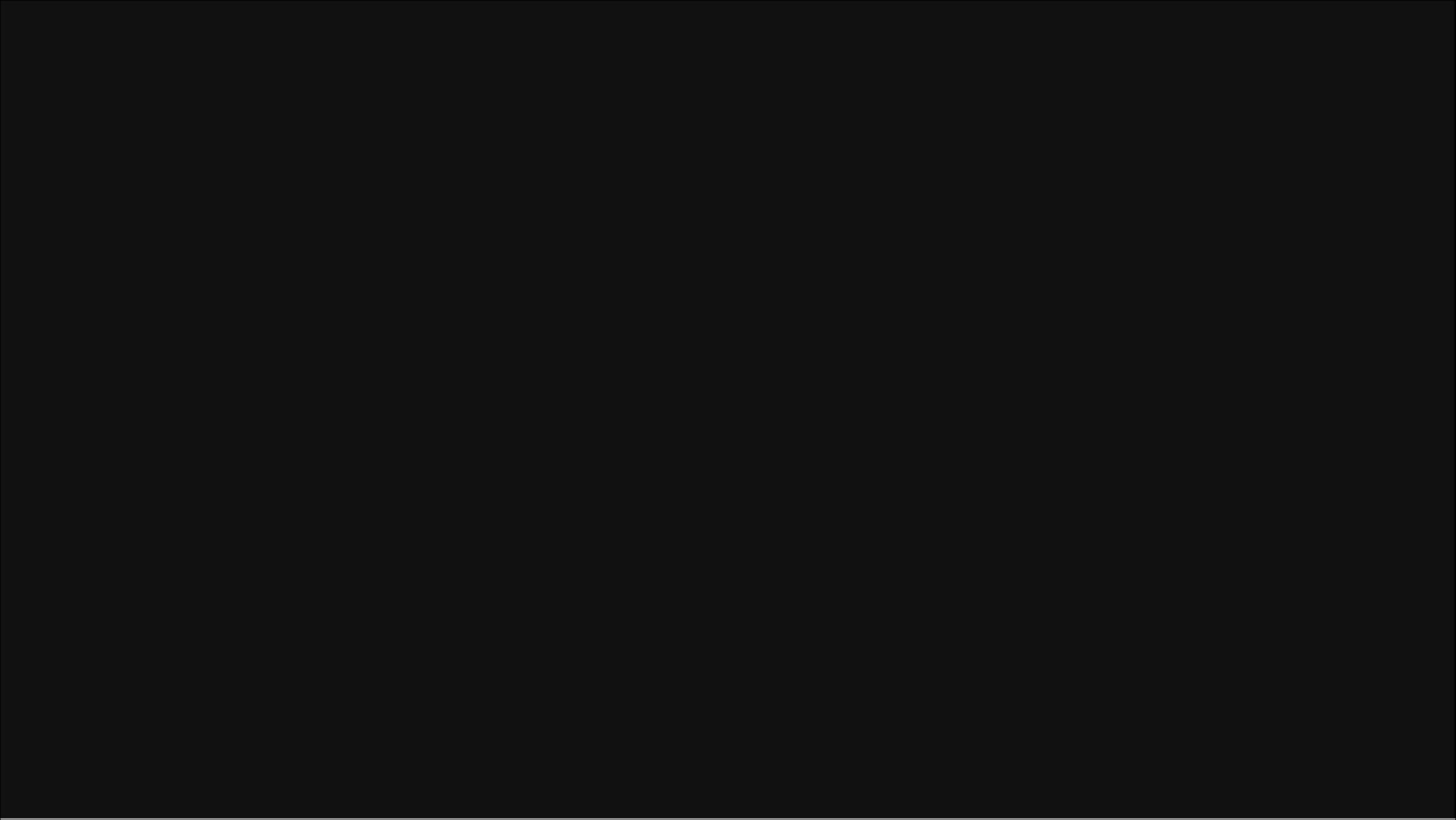
Strangling the monolith



Réarchitecturer

Branch-by-abstraction





Notre exemple

Jeu du Trivia



Notre exemple

Jeu du Trivia

🚀 Jeu de questions réponses



Notre exemple

Jeu du Trivia

🚀 Jeu de questions réponses

🔧 Code complexe et fragile



Notre exemple

Jeu du Trivia

🚀 Jeu de questions réponses

🔧 Code complexe et fragile

✨ Compréhension difficile



Refactorer dans du code legacy

Recette étape par étape

- 📢 Identifier les hotspots
- 🔨 Casser les dépendances
- 🧪 Ecrire les tests
- 🔧 Effectuer un changement
- 🧩 Refactorer

Identifier les hotspots

Quand ? vous ne savez pas par où commencer, votre temps est limité --> ROI

Comment ?

1. Calculer la Complexité du Code (fichiers SonarQube)

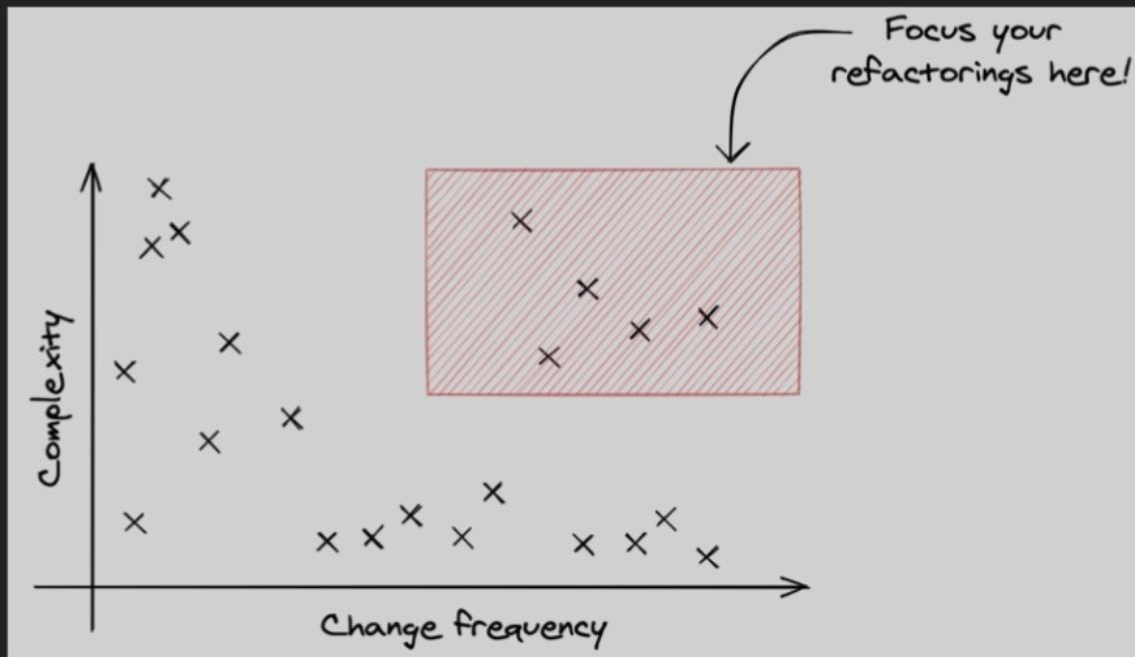
2. Calculez le Churn

```
git log --format=format: --name-only --since=12.month \  
| egrep -v '^$' \  
| egrep -v '\\.json$' \  
| sort \  
| uniq -c \  
| sort -nr \  
| head -50
```

3. Meilleur ROI = Complexité (Important) * Churn (Urgent) Augmente votre vitesse

Refactorer dans du code legacy

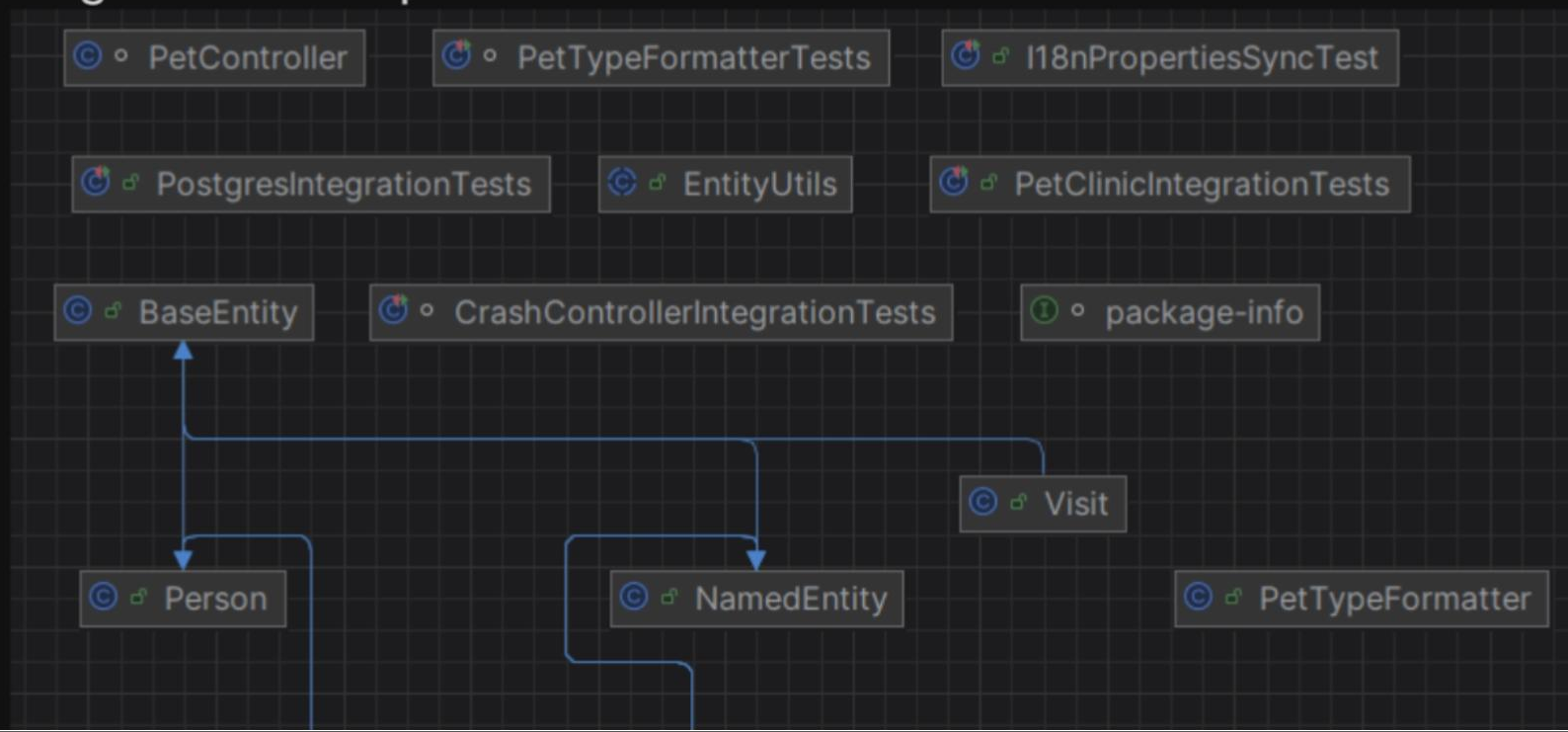
Focus sur la **valeur** métier



Identifier les hotspots

Quand ? vérifier les impacts d'un changement

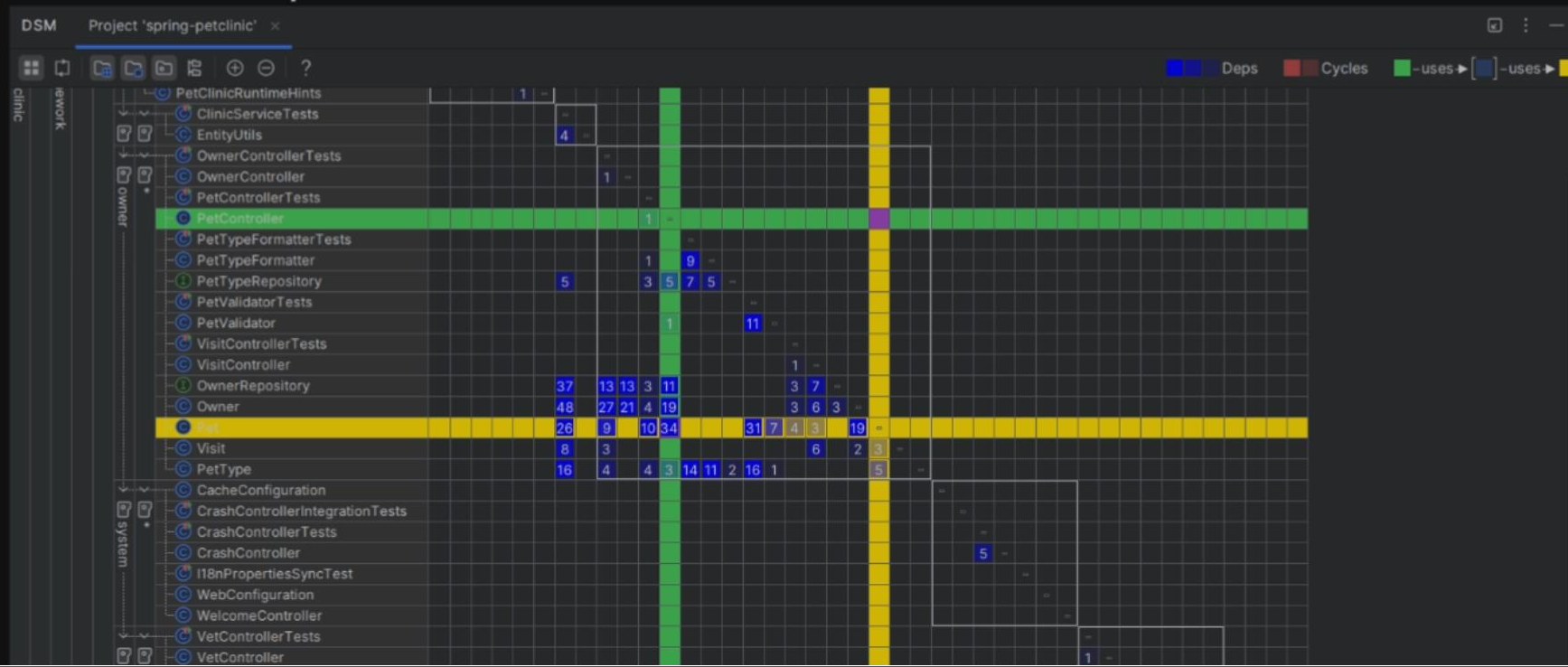
Diagramme de dépendance



Identifier les hotspots

Quand ? vérifier les impacts d'un changement

Matrice de dépendance



Découvrir du code legacy

Scratch Refactoring + Refactoring Exploratoire

Quand ? Vous essayez de comprendre ce que le code fait vraiment

Pratique >> Analyse théorique



Permet ? Big Picture

Scratch Refactoring

Ce qu'on a appris

- Deux points d'entrée 'public': `add()` et `roll()`
- `add()`: initialize player data + affichage
- `roll()`: determine si le joueur est piégé ou non ET le déplace



Tester

Approval Tests

Quand ? Mon code n'a aucun tests, je ne sais pas ce qu'il fait.

E2E $\geq$ Approval Tests $\geq$ Unit Test

<https://approvaltests.com/>

Approval Tests

Quand ? Mon code n'a aucun tests, je ne sais pas ce qu'il fait.

E2E $\geq$ Approval Tests $\geq$ Unit Test

<https://approvaltests.com/>

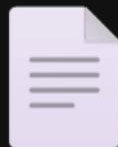


Approval Tests

Quand ? Mon code n'a aucun tests, je ne sais pas ce qu'il fait.

E2E $\geq$ Approval Tests $\geq$ Unit Test

<https://approvaltests.com/>



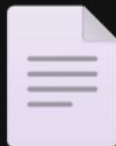
File
Received

Approval Tests

Quand ? Mon code n'a aucun tests, je ne sais pas ce qu'il fait.

E2E $\geq$ Approval Tests $\geq$ Unit Test

<https://approvaltests.com/>



File
Received

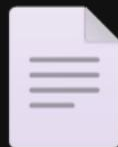


Approval Tests

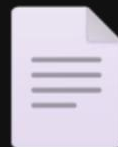
Quand ? Mon code n'a aucun tests, je ne sais pas ce qu'il fait.

E2E $\geq$ Approval Tests $\geq$ Unit Test

<https://approvaltests.com/>



File
Received



File
Approved

“ je dois **tester** avant de refactor,
mais mon code est **intestable** ”

Découpler : Core vs Infrastructure

Subclass & Override

// TODO: regarder "Passage devlille 2025 > conférences sur les tests IT" pour améliorer cette conf //

<https://blog.takima.fr/mon-passage-au-devlille-2025/>

Quand ? le code existant (non testé) a des effets de bords empêchant d'écrire des tests (appels BDD, ...).

Solution: extraire l'infrastructure (ex: logs, appels BDD) dans des méthodes dédiées.

```
1 // Before Subclass & Override
2 public boolean add(String playerName) {
3
4     players.add(playerName);
5     places[howManyPlayers()] = 0;
6     purses[howManyPlayers()] = 0;
7     inPenaltyBox[howManyPlayers()] = false;
8
9     System.out.println(playerName + " was added");
10    System.out.println("They are player number " + players.size());
11
12    return true;
13 }
```

Subclass & Override

// TODO: regarder "Passage devlille 2025 > conférences sur les tests IT" pour améliorer cette conf //

<https://blog.takima.fr/mon-passage-au-devlille-2025/>

Quand ? le code existant (non testé) a des effets de bords empêchant d'écrire des tests (appels BDD, ...).

Solution: extraire l'infrastructure (ex: logs, appels BDD) dans des méthodes dédiées.

```
1 // Before Subclass & Override
2 public boolean add(String playerName) {
3
4     players.add(playerName);
5     places[howManyPlayers()] = 0;
6     purses[howManyPlayers()] = 0;
7     inPenaltyBox[howManyPlayers()] = false;
8
9     System.out.println(playerName + " was added");
10    System.out.println("They are player number " + players.size());
11
12    return true;
13 }
```

Subclass & Override

// TODO: regarder "Passage devlille 2025 > conférences sur les tests IT" pour améliorer cette conf //

<https://blog.takima.fr/mon-passage-au-devlille-2025/>

Quand ? le code existant (non testé) a des effets de bords empêchant d'écrire des tests (appels BDD, ...).

Solution: extraire l'infrastructure (ex: logs, appels BDD) dans des méthodes dédiées.

```
1 // 1. Extract Variable sur le premier argument du println
2 public boolean add(String playerName) {
3
4     players.add(playerName);
5     places[howManyPlayers()] = 0;
6     purses[howManyPlayers()] = 0;
7     inPenaltyBox[howManyPlayers()] = false;
8
9     String message = playerName + " was added";
10    System.out.println(message);
11    System.out.println("They are player number " + players.size());
12
13    return true;
14 }
```

Subclass & Override

// TODO: regarder "Passage devlille 2025 > conférences sur les tests IT" pour améliorer cette conf //

<https://blog.takima.fr/mon-passage-au-devlille-2025/>

Quand ? le code existant (non testé) a des effets de bords empêchant d'écrire des tests (appels BDD, ...).

Solution: extraire l'infrastructure (ex: logs, appels BDD) dans des méthodes dédiées.

```
1 // 2. Extract Method sur le premier println
2 public boolean add(String playerName) {
3
4     players.add(playerName);
5     places[howManyPlayers()] = 0;
6     purses[howManyPlayers()] = 0;
7     inPenaltyBox[howManyPlayers()] = false;
8
9     String message = playerName + " was added";
10    log(message)
11    System.out.println("They are player number " + players.size());
12
13    return true;
14 }
15
```

Subclass & Override

// TODO: regarder "Passage devlille 2025 > conférences sur les tests IT" pour améliorer cette conf //

<https://blog.takima.fr/mon-passage-au-devlille-2025/>

Quand ? le code existant (non testé) a des effets de bords empêchant d'écrire des tests (appels BDD, ...).

Solution: extraire l'infrastructure (ex: logs, appels BDD) dans des méthodes dédiées.

```
1 // 3. Inline variable sur la variable message
2 public boolean add(String playerName) {
3
4     players.add(playerName);
5     places[howManyPlayers()] = 0;
6     purses[howManyPlayers()] = 0;
7     inPenaltyBox[howManyPlayers()] = false;
8
9     log(playerName + " was added");
10    System.out.println("They are player number " + players.size());
11
12    return true;
13 }
14
15 private void log(String message) {
```


Subclass & Override

// TODO: regarder "Passage devlille 2025 > conférences sur les tests IT" pour améliorer cette conf //

<https://blog.takima.fr/mon-passage-au-devlille-2025/>

Quand ? le code existant (non testé) a des effets de bords empêchant d'écrire des tests (appels BDD, ...).

Solution: extraire l'infrastructure (ex: logs, appels BDD) dans des méthodes dédiées.

```
1 // 4. Refactoring manuel (ok car on peut 'rollback' avec un Inline Method sur la méthode modifiée)
2 public boolean add(String playerName) {
3
4     players.add(playerName);
5     places[howManyPlayers()] = 0;
6     purses[howManyPlayers()] = 0;
7     inPenaltyBox[howManyPlayers()] = false;
8
9     log(playerName + " was added");
10    log("They are player number " + players.size());
11
12    return true;
13 }
14
15 private void log(String message) {
```

Subclass & Override

// TODO: regarder "Passage devlille 2025 > conférences sur les tests IT" pour améliorer cette conf //

<https://blog.takima.fr/mon-passage-au-devlille-2025/>

Quand ? le code existant (non testé) a des effets de bords empêchant d'écrire des tests (appels BDD, ...).

Solution: extraire l'infrastructure (ex: logs, appels BDD) dans des méthodes dédiées.

```
1 // 5. On expose log() en protected pour les tests
2 public boolean add(String playerName) {
3
4     players.add(playerName);
5     places[howManyPlayers()] = 0;
6     purses[howManyPlayers()] = 0;
7     inPenaltyBox[howManyPlayers()] = false;
8
9     log(playerName + " was added");
10    log("They are player number " + players.size());
11
12    return true;
13 }
14
15 protected void log(String message) {
```

Subclass & Override

// TODO: regarder "Passage devlille 2025 > conférences sur les tests IT" pour améliorer cette conf //

<https://blog.takima.fr/mon-passage-au-devlille-2025/>

Quand ? le code existant (non testé) a des effets de bords empêchant d'écrire des tests (appels BDD, ...).

Solution: extraire l'infrastructure (ex: logs, appels BDD) dans des méthodes dédiées.

```
1 // 6. Subclass Game pour les tests
2 class TestableGame extends Game {
3     protected log(message: string): void {
4         /* Override: doing nothing.. */
5     }
6 }
```

Subclass & Override

// TODO: regarder "Passage devlille 2025 > conférences sur les tests IT" pour améliorer cette conf //

<https://blog.takima.fr/mon-passage-au-devlille-2025/>

Quand ? le code existant (non testé) a des effets de bords empêchant d'écrire des tests (appels BDD, ...).

Solution: extraire l'infrastructure (ex: logs, appels BDD) dans des méthodes dédiées.

```
1 // 6. Subclass Game pour les tests
2 class TestableGame extends Game {
3     protected log(message: string): void {
4         /* Override: doing nothing.. */
5     }
6 }
```

Subclass & Override

// TODO: regarder "Passage devlille 2025 > conférences sur les tests IT" pour améliorer cette conf //

<https://blog.takima.fr/mon-passage-au-devlille-2025/>

Quand ? le code existant (non testé) a des effets de bords empêchant d'écrire des tests (appels BDD, ...).

Solution: extraire l'infrastructure (ex: logs, appels BDD) dans des méthodes dédiées.

```
1 // 6. Subclass Game pour les tests
2 class TestableGame extends Game {
3     protected log(message: string): void {
4         /* Override: doing nothing.. */
5     }
6 }
```

Seam : endroit où on peut rajouter un morceau de code sans modifier le code existant

Move Function to Delegate

Quand ? Après Subclass & Override. Améliorer le design du code à partir des tests.

```
1  class Game {
2      public boolean add(String playerName) {
3
4          players.add(playerName);
5          places[howManyPlayers()] = 0;
6          purses[howManyPlayers()] = 0;
7          inPenaltyBox[howManyPlayers()] = false;
8
9          log(playerName + " was added");
10         log("They are player number " + players.size());
11
12         return true;
13     }
14
15     protected void log(String message) {
16         System.out.println(message);
17     }
18
19     class TestableGame extends Game {
20         protected log(String message): void {
21             /* Override: doing nothing.. */
22         }
23     }
```

Move Function to Delegate

Quand ? Après Subclass & Override. Améliorer le design du code à partir des tests.

```
1  class Game {
2      public boolean add(String playerName) {
3
4          players.add(playerName);
5          places[howManyPlayers()] = 0;
6          purses[howManyPlayers()] = 0;
7          inPenaltyBox[howManyPlayers()] = false;
8
9          log(playerName + " was added");
10         log("They are player number " + players.size());
11
12         return true;
13     }
14
15     protected void log(String message) {
16         System.out.println(message);
17     }
18
19     class TestableGame extends Game {
20         protected log(String message): void {
21             /* Override: doing nothing.. */
22         }
23     }
```

Move Function to Delegate

Quand ? Après Subclass & Override. Améliorer le design du code à partir des tests.

```
1 // 1. Créer une interface qu'on aimerait utiliser: Logger
2 interface Logger {
3     void log(String message);
4 }
```


Move Function to Delegate

Quand ? Après Subclass & Override. Améliorer le design du code à partir des tests.

```
1 // 2. Créer une interface qu'on aimerait utiliser: Logger
2 interface Logger {
3     void log(String message);
4 }
5
6 class ConsoleLogger implements Logger {
7     void log(String message) {
8
9     }
10 }
```

Move Function to Delegate

Quand ? Après Subclass & Override. Améliorer le design du code à partir des tests.

```
1 // 3. Move Method (ou Move Function) refactoring
2 class Game {
3     public boolean add(String playerName) {
4         private Logger logger = new ConsoleLogger();
5
6         players.add(playerName);
7         places[howManyPlayers()] = 0;
8         purses[howManyPlayers()] = 0;
9         inPenaltyBox[howManyPlayers()] = false;
10
11         log(playerName + " was added");
12         log("They are player number " + players.size());
13
14         return true;
15     }
16
17     protected void log(String message) {
18         System.out.println(message);
19     }
20
21     class TestableGame extends Game {
22         protected log(String message): void {
23             /* Override: doing nothing.. */
24         }
25     }
26 }
```

Move Function to Delegate

Quand ? Après Subclass & Override. Améliorer le design du code à partir des tests.

```
1 // 3. Move Method (ou Move Function) refactoring
2 class Game {
3     public boolean add(String playerName) {
4         private Logger logger = new ConsoleLogger();
5
6         players.add(playerName);
7         places[howManyPlayers()] = 0;
8         purses[howManyPlayers()] = 0;
9         inPenaltyBox[howManyPlayers()] = false;
10
11         log(playerName + " was added");
12         log("They are player number " + players.size());
13
14         return true;
15     }
16
17     protected void log(String message) {
18         System.out.println(message);
19     }
20
21     class TestableGame extends Game {
22         protected log(String message): void {
23             /* Override: doing nothing.. */
24         }
25     }
26 }
```

Move Function to Delegate

Quand ? Après Subclass & Override. Améliorer le design du code à partir des tests.

```
1 // 4. Créer une interface qu'on aimerait utiliser: Logger
2 interface Logger {
3     void log(String message);
4 }
5
6 class ConsoleLogger implements Logger {
7     void log(String message) {
8         System.out.println(message);
9     }
10 }
```

Move Function to Delegate

Quand ? Après Subclass & Override. Améliorer le design du code à partir des tests.

```
1 // 5. Injecter le 'Logger' Delegate dans le constructeur
2 class Game {
3     public boolean add(String playerName) {
4         private Logger logger = new ConsoleLogger();
5     }
6     // code..
```

Move Function to Delegate

Quand ? Après Subclass & Override. Améliorer le design du code à partir des tests.

```
1 // 5. Injecter le 'Logger' Delegate dans le constructeur
2 class Game {
3     public boolean add(String playerName) {
4         public Game(Logger logger) {
5             logger = new ConsoleLogger();
6         }
7         // code..
```

Move Function to Delegate

Quand ? Après Subclass & Override. Améliorer le design du code à partir des tests.

```
1 // 4. Créer une interface qu'on aimerait utiliser: Logger
2 interface Logger {
3     void log(String message);
4 }
5
6 class ConsoleLogger implements Logger {
7     void log(String message) {
8         System.out.println(message);
9     }
10 }
```

Move Function to Delegate

Quand ? Après Subclass & Override. Améliorer le design du code à partir des tests.

```
1 // 5. Créer une implémentation dédiée à nos tests pour Logger
2 interface Logger {
3     void log(String message);
4 }
5
6 class ConsoleLogger implements Logger {
7     void log(String message) {
8         System.out.println(message);
9     }
10 }
11
12 // Pour les tests
13 class NoopLogger implements Logger {
14     void log(String message) { /* NoOp = ne fait rien */ }
15 }
```


Move Function to Delegate

Quand ? Après Subclass & Override. Améliorer le design du code à partir des tests.

```
1 class TestableGame extends Game {  
2     protected log(String message): void {  
3         /* Override: doing nothing.. */  
4     }  
5 }
```

Move Function to Delegate

Quand ? Après Subclass & Override. Améliorer le design du code à partir des tests.

```
1 class TestableGame extends Game {  
2     public TestableGame() {  
3         super(new NoopLogger()); // Laisse l'instance de logger gérer le comportement  
4     }  
5 }
```

Refactorer dans du code legacy

A retenir

Refactorer dans du code legacy

A retenir

Stratégie non prédictive

Refactorer dans du code legacy

A retenir

Stratégie non prédictive

- Fail-fast

Refactorer dans du code legacy

A retenir

Stratégie non prédictive

- Fail-fast
- Scratch refactoring et exploratoire

Refactorer dans du code legacy

A retenir

Stratégie non prédictive

- Fail-fast
- Scratch refactoring et exploratoire
- Test-first

Refactorer dans du code legacy

A retenir

Stratégie non prédictive

- Fail-fast
- Scratch refactoring et exploratoire
- Test-first
- Méthode Mikado

Refactorer dans du code legacy

A retenir

Stratégie non prédictive

- Fail-fast
- Scratch refactoring et exploratoire
- Test-first
- Méthode Mikado
- Over-committing (minuteur: N minutes) ou `git add -p`

Refactorer dans du code legacy

A retenir

Stratégie non prédictive

- Fail-fast
- Scratch refactoring et exploratoire
- Test-first
- Méthode Mikado
- Over-committing (minuteur: N minutes) ou `git add -p`

Besoin d'un filet de sécurité efficace

Refactorer dans du code legacy

A retenir

Stratégie non prédictive

- Fail-fast
- Scratch refactoring et exploratoire
- Test-first
- Méthode Mikado
- Over-committing (minuteur: N minutes) ou `git add -p`

Besoin d'un filet de sécurité efficace

- Approval testing

Refactorer dans du code legacy

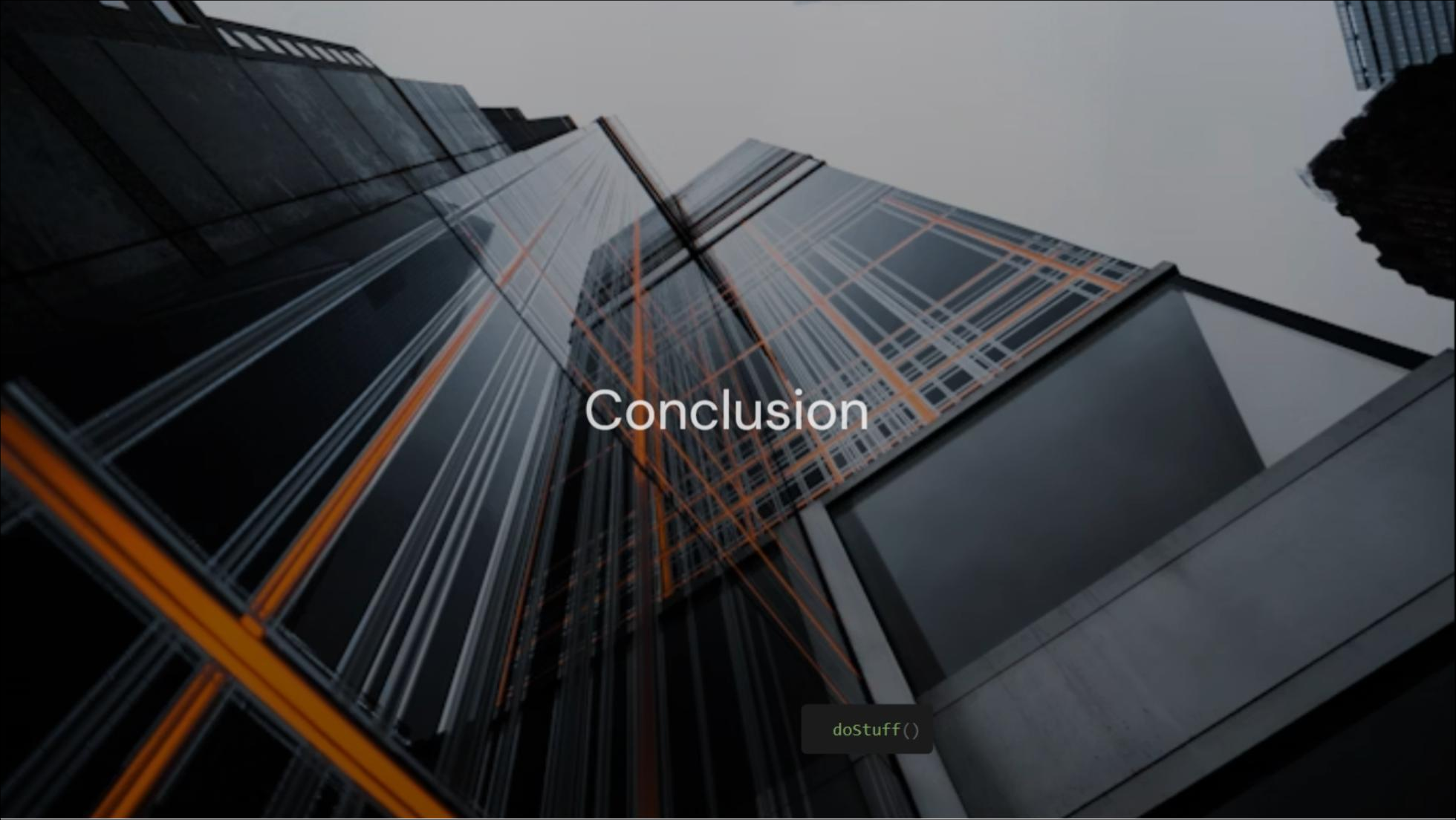
A retenir

Stratégie non prédictive

- Fail-fast
- Scratch refactoring et exploratoire
- Test-first
- Méthode Mikado
- Over-committing (minuteur: N minutes) ou `git add -p`

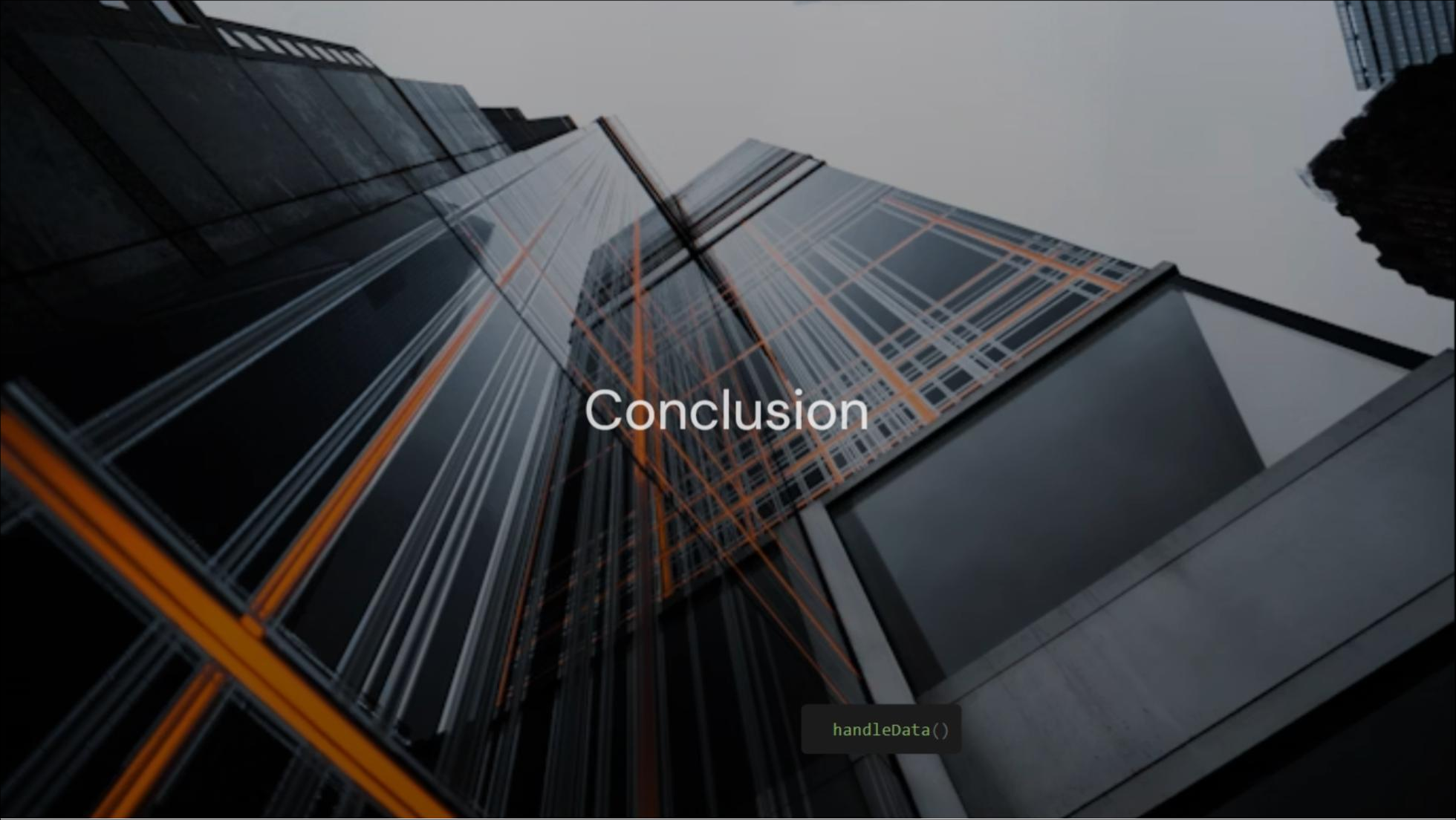
Besoin d'un filet de sécurité efficace

- Approval testing
- Mutation testing

A low-angle, upward-looking shot of several modern skyscrapers with glass facades. The perspective creates a sense of height and scale. Overlaid on the image is a grid of thin orange lines that form a pattern across the buildings. The sky is a uniform, overcast grey.

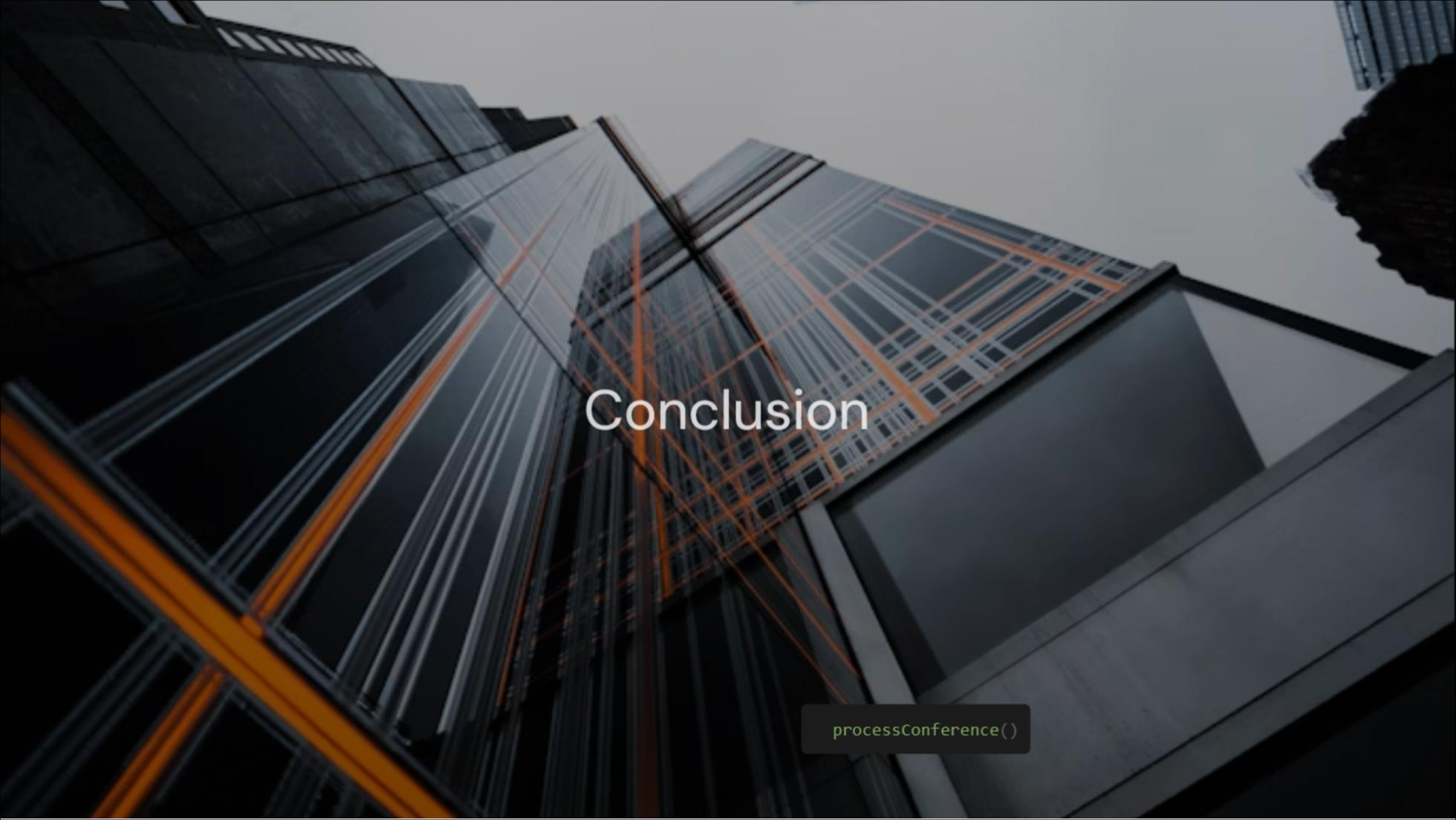
Conclusion

`doStuff()`

A low-angle, upward-looking shot of several modern skyscrapers with glass facades. The perspective creates a sense of height and scale. Overlaid on the image is a grid of orange lines that form a pattern across the buildings, suggesting a data visualization or architectural overlay.

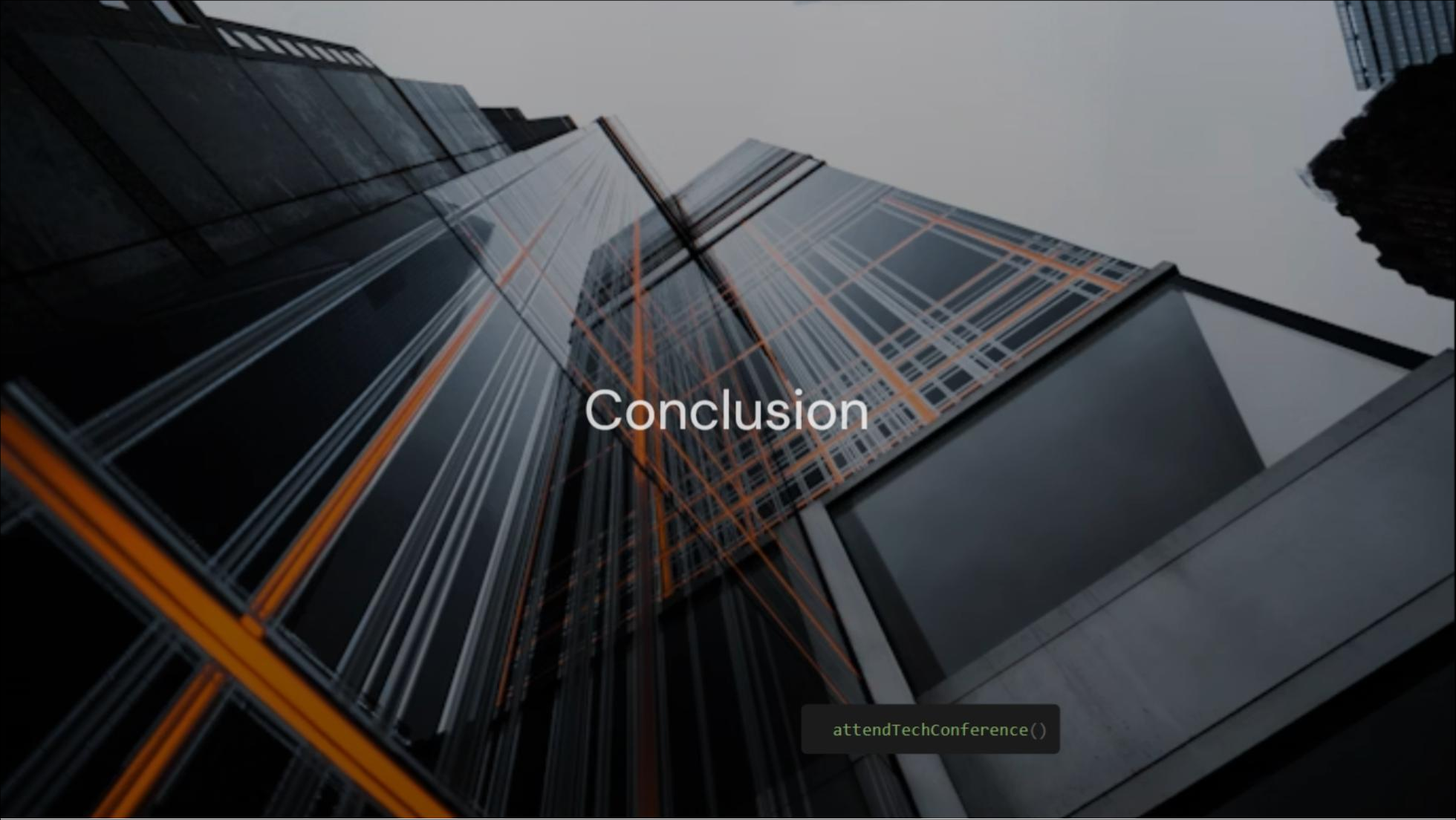
Conclusion

```
handleData()
```


A low-angle, upward-looking photograph of several modern skyscrapers with glass facades. The perspective creates a sense of height and scale. Overlaid on the image is a grid of thin orange lines that form a pattern across the buildings. The sky is a pale, overcast grey.

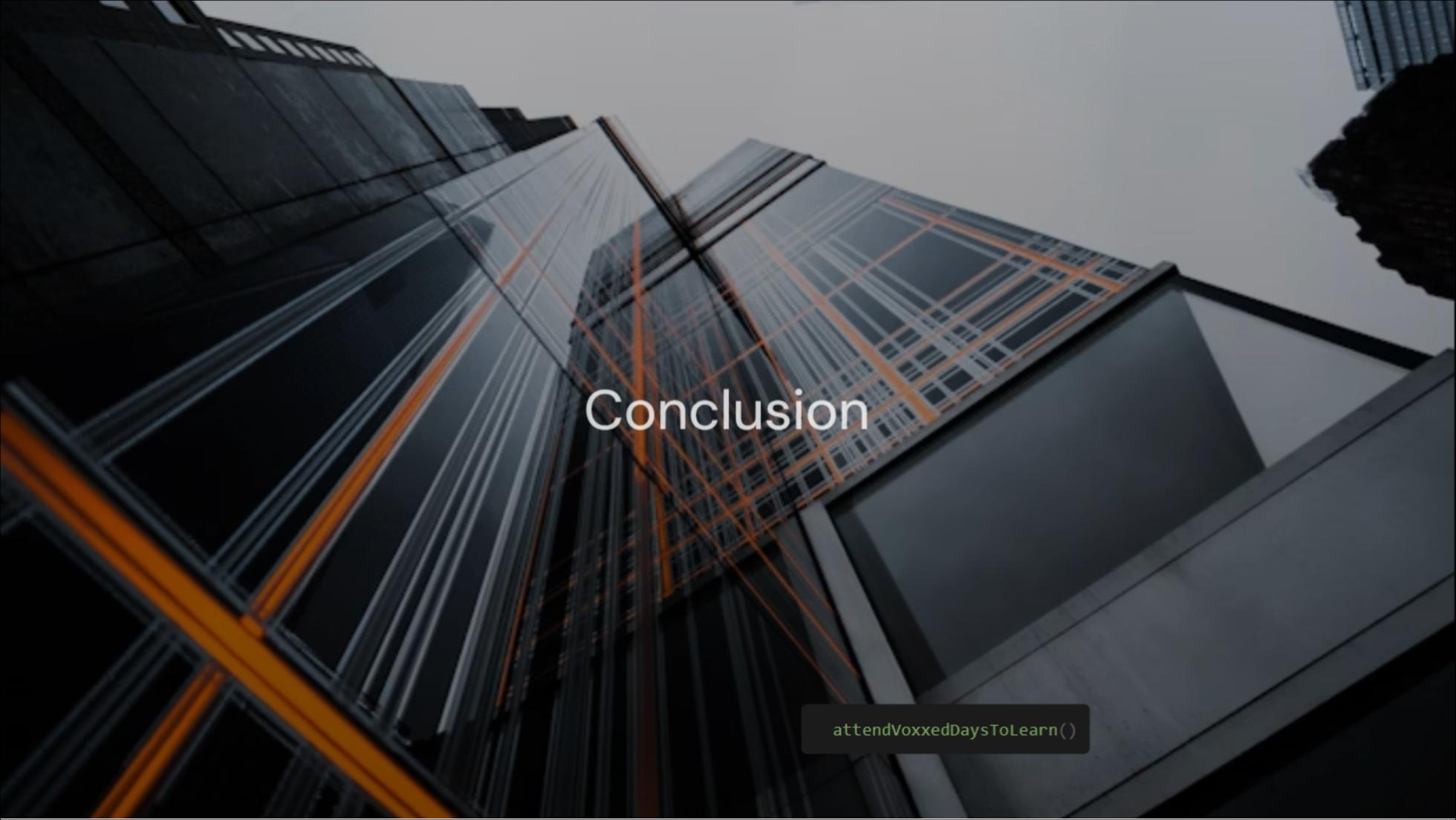
Conclusion

```
processConference()
```

A low-angle, upward-looking shot of several modern skyscrapers with glass facades. The perspective creates a sense of height and scale. Overlaid on the image is a grid of thin orange lines that form a pattern across the buildings. The sky is a pale, overcast grey.

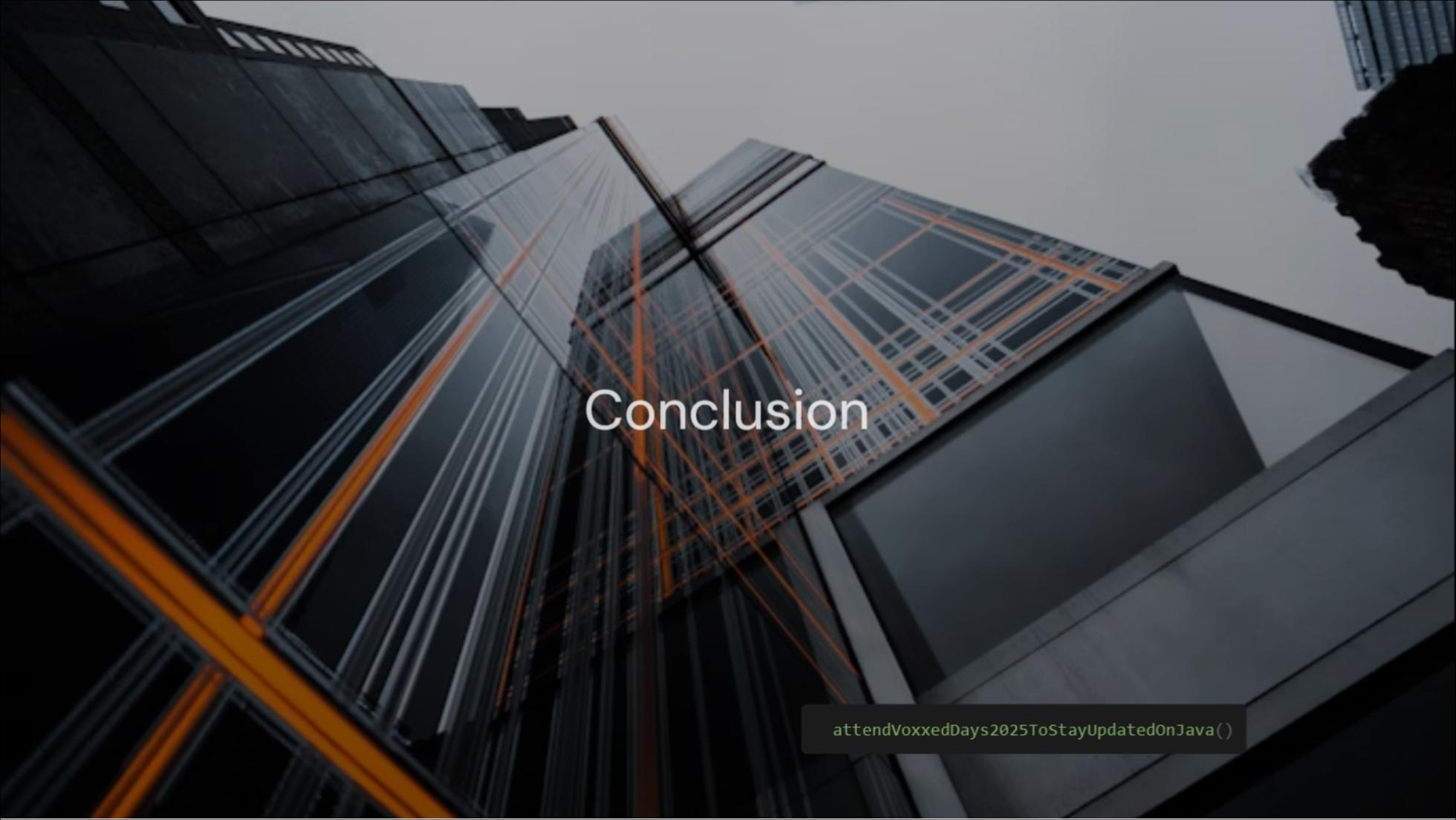
Conclusion

```
attendTechConference()
```


A low-angle, upward-looking photograph of several modern skyscrapers. The buildings are constructed with dark glass and steel, reflecting the sky. Overlaid on the image is a grid of thin orange lines, creating a technical or architectural feel. The perspective makes the buildings appear to converge towards the top of the frame.

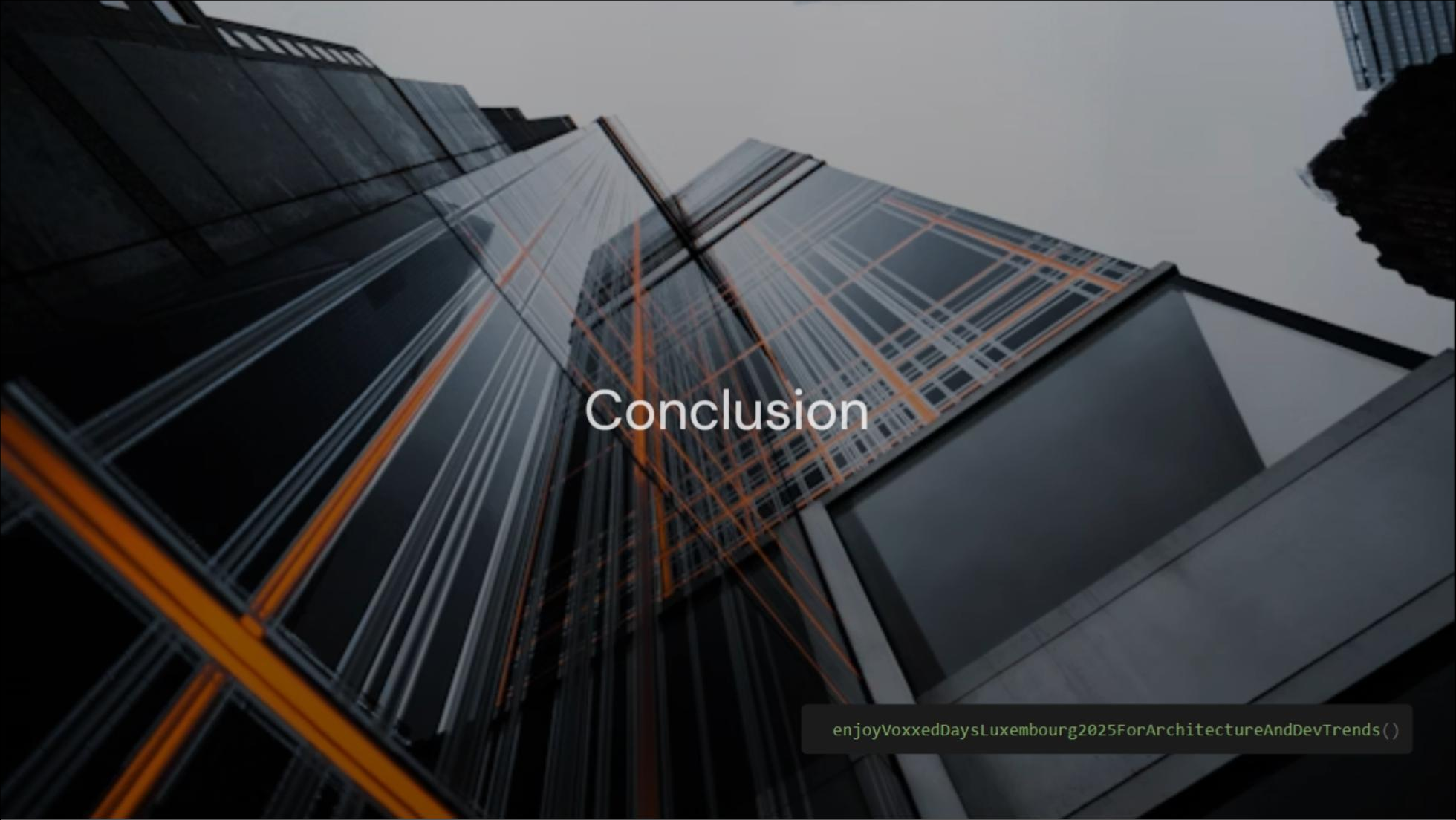
Conclusion

`attendVoxxedDaysToLearn()`

A low-angle, upward-looking shot of several modern skyscrapers with glass facades. The perspective creates a sense of height and scale. Overlaid on the image is a grid of thin orange lines that form a pattern across the buildings. The word "Conclusion" is centered in the middle of the image in a white, sans-serif font.

Conclusion

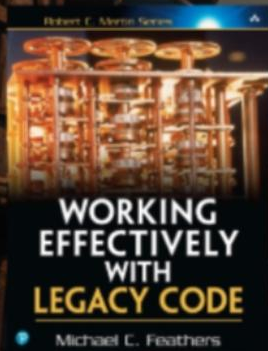
`attendVoxxedDays2025ToStayUpdatedOnJava()`

A low-angle, upward-looking photograph of several modern skyscrapers. The buildings feature glass facades and prominent orange structural elements, possibly scaffolding or architectural details, creating a complex geometric pattern of lines against a grey, overcast sky. The perspective makes the buildings appear to converge towards the top of the frame.

Conclusion

`enjoyVoxxedDaysLuxembourg2025ForArchitectureAndDevTrends()`

Quelques références



Legacy refactoring (katas)



Valentin Dumas

Ingénieur Logiciel



Merci de votre attention !



*Pour les exemples,
c'est là !*



